1. Write a C program to implement Merge sort algorithm for sorting a list of integers in ascending order.

```
int a[20];
void MergeSort(int,int);
void Merge(int,int,int);
void main()
{
        int i,n;
        printf("\n Enter array size:");
        scanf("%d",&n);
        printf("\nEnter array elements:");
        for(i=0;i<n;i++)
        scanf("%d",&a[i]);
        MergeSort(0,n-1);
        printf("\nArray elements after sorting:");
        for(i=0;i<n;i++)
          printf("%d ",a[i]);
}
void  MergeSort(int low, int high)
{
        int mid;
        if (low <high)
        {
                mid = (low + high)/2;
                MergeSort(low, mid);
                MergeSort(mid+1, high);
               Merge(low, mid, high);
    }
}
void Merge(int low, int mid, int high)
{
        int h, i, j, k, b[50];
        h = low;
        j = mid+1;
        i = low;
        while ((h <= mid) && (j <= high) )
        {
                if( a[h] <= a[j] )
                {
                        b[ i ] = a[ h ];
                        h = h+1;
                }
                else
                {
                       b[ i ] = a[ j ];
                        j = j+1;
                }
                i = i+1;
        }
         if( h > mid )
                for ( k = j ; k <= high; k++ )
                {
                        b[ i ] = a[k]; i = i+1;
                }
        else
                for ( k = h ; k <= mid; k++ )
```

```
                {
                        b[ i ]  = a[ k ]; i = i+1;
                }
        for( k = low; k <= high; k++ ) // copy all the elements back to array A.
                a[ k ] = b[ k ] ;
}
```

2. Write a C program to implement Quick Sort algorithm for sorting a list of integers in ascending order.

```
int a[20];
void QuickSort(int, int);
int Partition(int, int);
void Interchange(int, int);
void main()
{
        int i,n;
        printf("\n Enter array size:");
        scanf("%d",&n);
        printf("\nEnter array elements:");
        for(i=0;i<n;i++)
           scanf("%d",&a[i]);
        QuickSort(0,n-1);
        printf("\nArray elements after sorting:");
        for(i=0;i<n;i++)
           printf("%d ",a[i]);
}
void QuickSort(int low, int high)
{
        int j;
        if (low<high)
        {
              j = Partition(low, high);
             QuickSort(low,j-1);
             QuickSort(j+1,high);
        }
}
int Partition(int low, int high)
{
        int pivot, i, j;
        pivot = a[low];
        i =low;
        j= high+1;
        while(i < j)
        {
                i++;
                while(a[ i ] < pivot)      // while(a[ i ] < pivot && i<=high) use it when given array elements
                      i++;                 // are in decreasing order
                j--;
                while(a[ j ]  > pivot)      // while(a[ j ] > pivot && j>=low) use it when given array elements
                       j--;                 // are in decreasing order
               if(i < j)
                    Interchange(i, j);
   }
   Interchange(low, j);
```

```
    return j;
}
void Interchange (int x,int y)
{
          int temp;
          temp=a[x];
         a[x]=a[y];
          a[y]=temp;
}
```

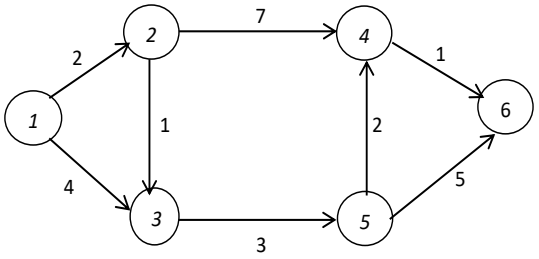
3. Write a C program to implement Dijkstra's algorithm for the single source shortest path problem.

```
int cost[10][10];
void dijkstra(int,int);
void main()
{
  int i,j,n,u;
  printf("Enter no. of vertices:");
  scanf("%d",&n);
  printf("\nEnter the cost matrix:\nEnter 9999 if there is no edge between the vertices\n");
  for(i=0;i<n;i++)
    for(j=0;j<n;j++)
      scanf("%d",&cost[i][j]);
  printf("\nEnter the starting node:");
  scanf("%d",&u);
  dijkstra(n,u);
}
void dijkstra(int n,int startnode)
{
   int distance[10];
   int visited[10],mindistance,nextnode,i,j,count;
   for(i=0;i<n;i++)
   {
     distance[i]=cost[startnode][i];
     visited[i]=0;
   }
   distance[startnode]=0;
   visited[startnode]=1;
   count=1;
   while(count<n-1)
   {
      mindistance=9999; //INFINITY 9999
      for(i=0;i<n;i++)
        if(distance[i]<mindistance&&!visited[i])
        {
              mindistance=distance[i];
            nextnode=i;
        }
      visited[nextnode]=1;
      for(i=0;i<n;i++)
        if(!visited[i])
          if(mindistance+cost[nextnode][i]<distance[i])
          {
               distance[i]=mindistance+cost[nextnode][i];
          }
      count++;
   }
```

```
    for(i=0;i<n;i++)
      if(i!=startnode)
        printf("\nDistance from node%d to node%d=%d",startnode,i,distance[i]);
}
```

Output:

```
Enter no. of vertices: 6

Enter the cost matrix:
Enter 9999 if there is no edge between the vertices
9999 2 4 9999 9999 9999
9999 9999 1 7 9999 9999
9999 9999 9999 9999 3 9999
9999 9999 9999 9999 9999 1
9999 9999 9999 2 9999 5
9999 9999 9999 9999 9999 9999
```




```
Enter the starting node: 0

Distance from node0 to node1=2
Distance from node0 to node2=3
Distance from node0 to node3=8
Distance from node0 to node4=6
Distance from node0 to node5=9
```

4. Write a C program to implement Prim's algorithm to generate a minimum cost spanning tree.

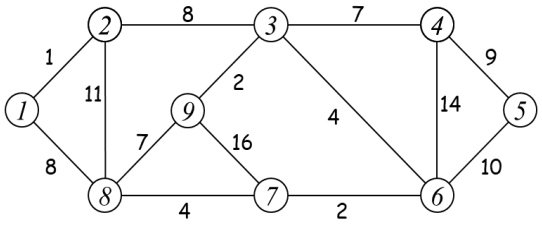
```
int n, cost[10][10];
void prim();
void main()
{
        int i, j;
        printf("\nEnter the no. of vertices:");
        scanf("%d", &n);
        printf("Enter the cost matrix:\n");
        for (i = 0; i< n; i++)
                for (j = 0; j < n; j++)
                {
                        scanf("%d", &cost[i][j]);
                }
        prim();
}
void prim()
{
        int i, j, startVertex, endVertex;
        int k, nearr[10], temp, minimumCost = 0, tree[10][2];

       /*Initialize nearrarray to -1
       for (i = 0; i< n; i++)
           nearr[i] = -1;

        /*For first smallest edge */
          temp = cost[0][0];
          for (i = 0; i< n; i++)
          {
                  for (j = 0; j < n; j++)
                  {
                          if (cost[i][j] < temp)
                          {
```

```
                    temp = cost[i][j];
                    startVertex = i;
                     endVertex = j;
                }
            }
    }
    /* Now we have fist smallest edge in graph */
    tree[0][0] = startVertex;
    tree[0][1] = endVertex;
    minimumCost = cost[startVertex][endVertex];

    /* To indicate included vertices initialize nearr[] for them to 100 */
    nearr[startVertex] = 100;
    nearr[endVertex] = 100;

    /* Now initialize nearr[ ] */

for (i = 0; i< n; i++)
{
       if(nearr[i]!=100)
            if(cost[i][startVertex] < cost[i][endVertex])
                nearr[i] = startVertex;
            else
                nearr[i] = endVertex;
}

/* Now find out remaining n-2 edges */
temp = 999;
for (i = 1; i< n - 1; i++)   // outer for loop starts
{
            for (j = 0; j < n; j++)
            {
                if (nearr[j] != 100 && cost[j][nearr[j]] < temp)
                {
                    temp = cost[j][nearr[j]];
                    k = j;
                }
            }

            /* Now we have got next smallest edge */
            tree[i][0] = k;
            tree[i][1] = nearr[k];
            minimumCost = minimumCost + cost[k][nearr[k]];
            nearr[k] = 100;
            /* Now find if k is nearest to any vertex than its previous near value */

            for (j = 0; j < n; j++)
            {
                if (nearr[j] != 100 && cost[j][nearr[j]] > cost[j][k])
                    nearr[j] = k;
            }
            temp = 999;
}  /* outer for loop ends */

/* Now we have the answer, just print it */
printf("\nThe min spanning tree is \n");
for (i = 0; i< n - 1; i++)
```

```
    {
            for (j = 0; j < 2; j++)
                printf("%d ", tree[i][j]);
        printf("\n");
    }
    printf("\nMin cost: %d", minimumCost);
}
```

Output:

```
Enter the no. of vertices: 9
Enter the cost matrix:
Enter 999 if there is no edge between the vertices
999 1 999 999 999 999 999 8 999
1 999 8 999 999 999 999 11 999
999 8 999 7 999 4 999 999 2
999 999 7 999 9 14 999 999 999
999 999 999 9 999 10 999 999 999
999 999 4 14 10 999 2 999 999
999 999 999 999 999 2 999 4 16
8 11 999 999 999 999 4 999 7
999 999 2 999 999 999 16 7 999
The min spanning tree is
0 1
2 1
8 2
5 2
6 5
7 6
3 2
4 3
Min cost: 37
```



5. Write a C program to implement 0/1 knapsack problem using greedy method

```
#include <stdio.h>
int main()
{
  int i,n;
  float m,U,p[10],w[10],tprofit=0,x[10]={0};
  printf("\nEnter no.of items:");
  scanf("%d",&n);
  printf("\nEnter knapsack size:");
  scanf("%f",&m);
  printf("\nEnter profits in p[i]/w[i]>=p[i+1]/w[i+1] order:");
  for(i=0;i<n;i++)
    scanf("%f",&p[i]);
  printf("\nEnter weights in p[i]/w[i]>=p[i+1]/w[i+1] order:");
  for(i=0;i<n;i++)
    scanf("%f",&w[i]);
  U=m;
  for(i=0;i<n;i++)
  {
    if(w[i]>U) break;
    x[i]=1;
    U=U-w[i];
  }
  if(i<n)
    x[i]=U/w[i];
```

```
  for(i=0;i<n;i++)
    tprofit=tprofit+x[i]*p[i];
  printf("\nSolution is:");
  for(i=0;i<n;i++)
    printf("%f ",x[i]);
  printf("\nTotal profit:%f",tprofit);
  return 0;
}
```

Output:

```
Enter no.of items: 5
Enter knapsack size: 10
Enter profits in p[i]/w[i]>=p[i+1]/w[i+1] order: 50 40 30 32 12
Enter weights in p[i]/w[i]>=p[i+1]/w[i+1] order: 1 2 6 8 4
Solution is: 1.0, 1.0, 1.0, 0.125, 0.0
Total profit: 124.0
```

6. Write a C program to implement greedy algorithm for job sequencing with deadlines

```
#include<stdio.h>
int n, p[20],d[20],j[20];
int js(int[],int[],int);
void main()
{
int i, k, total;
printf("\nEnter no of jobs:");
scanf("%d", &n);
printf("Enter profit & dead line\n");
for(i=1; i<=n; i++)
{
scanf("%d%d", &p[i], &d[i]);
}
k = js( d, j, n );
printf("jobs after sorting\n");
printf("job_no profit deadline\n");
for(i=1;i<=n;i++)
{
printf("%d %d %d\n",i,p[i],d[i]);
}
printf("Jobs processing sequence for maximum profit\n");
printf("(Sequence is with respect to the sorted list)\n");
total=0;
for(i=1; i<=k; i++)
{
printf("J%d ", j[i]);
total = total + p[j[i]] ;
 }
printf("Total profit = %d", total);
} // main end

int js(int d[],int j[],int n)
{
int temp,i,k, r, q;
for(i=1; i<n; i++) // Sorting jobs in decreasing order of profit
{
for( k=1; k<n; k++)
{
```

```
        if( p[k] < p[k+1] )
        {
        temp = p[k];
        p[k] = p[k+1];
        p[k+1] = temp;
        temp = d[k];
        d[k] = d[k+1];
        d[k+1] = temp;
        }
}
}
d[0] = j[0] = 0;
j[1] = 1;
k = 1;
for(i=2; i<=n; i++)
{
r=k;
while(d[j[r]] > d[i] && d[j[r]] != r)
{
        r = r-1;
}
if(d[i] > r)
{
        for(q=k; q>=r+1; q--)
        {
        j[q+1] = j[q];
        }
        j[r+1] = i;
        k++;
}
} // for end

return k;
}
```

Output:

```
Enter no of jobs: 5
Enter profit & dead line
100 2
19 1
27 2
25 1
15 3
jobs after sorting
job_no profit deadline
1 100 2
2 27 2
3 25 1
4 19 1
5 15 3
Jobs processing sequence for maximum profit
(Sequence is with respect to the sorted list)
J1 J2 J5 Total profit = 142
```

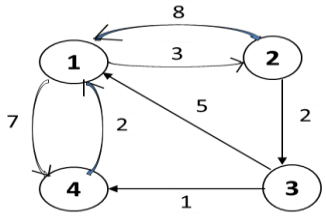
7. Write a C program to implement Floyd-Warshall algorithm for all pairs shortest path problem

```
void main()
{
```

```
    int n, c[10][10], i, j, k;
    printf("\nEnter number of vertices:");
    scanf("%d",&n);
    printf("\nEnter adjacency matrix");
    printf("\nEnter 0 for self-loop & enter 999 if no direct edge between two vertices\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d",&c[i][j]);
    for(k=0; k<n; k++)
    {
        for(i=0; i<n; i++)
        {
            for(j=0; j<n; j++)
            {
                if (c[i][k] + c[k][j] < c[i][j])
                    c[i][j] = c[i][k] + c[k][j];
            }
        }
    }
    printf("All pairs shortest path matrix is...\n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
            printf("%d ",c[i][j]);
        printf("\n");
    }
}
```

Output:

```
Enter number of vertices: 4
Enter adjacency matrix
Enter 0 for self-loop & enter 999 if no direct edge between two vertices
0 3 999 7
8 0 2 999
5 999 0 1
2 999 999 0
All pairs shortest path matrix is...
0 3 5 6
5 0 2 3
3 6 0 1
2 5 7 0
```



8. Write a C program to implement n-queens problem using backtracking

```
#include<stdio.h>
#include<stdlib.h>
int x[10],count=1;
void NQueens(int,int);
int place(int,int);
void main()
{
   int n;
   printf("Enter number of Queens:");
   scanf("%d",&n);
   NQueens(1,n);
}
void NQueens(int k, int n)
{
```

```
    int i,j;
    for(i=1;i<=n;i++)
    {
        if (place(k,i))
        {
            x[k]=i;
            if(k==n)
            {
                printf("Solution-%d:\n", count);
                for(j=1; j<=n;j++)
                    printf("Queen%d-[%d,%d] ",j,j,x[j]);
                printf("\n");
                count++;
            }
            else
                NQueens(k+1,n);
        }
    }
}
int place(int k,int i)
{
    int j;
    for(j=1; j<=k-1; j++)
    {
        if((x[j] == i) || ( abs(x[j]-i)==abs(j-k)))
        {
            return 0;
        }
    }
    return 1;
}
```

Output:

```
Enter number of Queens: 4
Solution-1:
Queen1-[1,2] Queen2-[2,4] Queen3-[3,1] Queen4-[4,3]
Solution-2:
Queen1-[1,3] Queen2-[2,1] Queen3-[3,4] Queen4-[4,2]
```